

ota-protocol

Field	Value
Revision	2
Created	2026-06-07
Status	implemented
Part of	Helix OTA
Module path	github.com/HelixDevelopment/ota-protocol
Language	go (1.26, stdlib only)
License	Apache-2.0

Overview

`ota-protocol` (package `otaprotocol`) is the **decoupled keystone** of the Helix OTA control plane: the single source of truth for the shared wire protocol. It contains ONLY pure contracts — typed enums, the request/response structs that mirror the REST `/api/v1` OpenAPI schemas, their JSON (de)serialization, and validation helpers. It has **no I/O, no HTTP, no database, no transport code** and depends on the Go standard library only (`doc.go`), so every other Helix OTA module — the server, the rollout engine, the validator, the device agents — can vendor it as one shared vocabulary.

The contracts mirror the committed specifications: `api/openapi.yaml` / `api/endpoints.md` (the REST surface) and `database/schema.md` (the enumerated relational domains, reproduced as Go enums).

Every enum implements `fmt.Stringer`, a `Valid()` method, and marshals to/from JSON as its lowercase string token; unknown tokens are **rejected** on `unmarshal` (`enums.go`).

Public API

Enums (`enums.go`) — each has `String()`, `Valid()`, `MarshalJSON`, `UnmarshalJSON`

- `OSType` — device/artifact OS family: `OSAndroid`, `OSLinux`, `OSWindows`, `OSOther`.
- `ArtifactType` — `ArtifactFull`, `ArtifactIncremental`, `ArtifactDelta`.
- `UploadStatus` — server-side validation outcome: `UploadPending`, `UploadValidated`, `UploadFailed`.

- `ReleaseChannel` — `ChannelStable`, `ChannelBeta`, `ChannelCanary`, `ChannelInternal`.
- `DeploymentStatus` — `DeploymentDraft`, `DeploymentActive`, `DeploymentPaused`, `DeploymentCompleted`, `DeploymentFailed`, `DeploymentCancelled`.
- `DeviceDeploymentStatus` — per-device lifecycle
incl. `DeviceDeployRolledBack` (A/B auto-rollback): `DeviceDeployPending`, `DeviceDeployDownloading`, `DeviceDeployInstalling`, `DeviceDeployVerifying`, `DeviceDeploySuccess`, `DeviceDeployFailed`, `DeviceDeployRolledBack`.
- `TelemetryEvent` — the EXACTLY six device-reported events:
`EventDownloadStarted`, `EventInstalling`, `EventInstalled`, `EventVerifying`, `EventSuccess`, `EventFailure`.
- `TargetType` — deployment target selector: `TargetAll`, `TargetGroup`, `TargetDevice`.

Wire structs (`types.go`)

- `ArtifactMeta` — integrity + targeting metadata for an upload (SHA256/SHA512, size, `OSType`, board, version, signature, payload properties).
- `Artifact` — stored-and-verified artifact metadata returned by the artifact endpoints.
- `Release` — binds a validated artifact to a published, deployable version on a channel.
- `DeviceRegistrationRequest` / `DeviceRegistrationResponse` — provision a device and mint its device-scoped bearer token.
- `UpdateCheckRequest` — the optional client-reported `current_version`.
- `UpdateAvailable` — the 200 apply instruction for `update_engine.applyPayload(url, offset, size, headers)`.
- `UpdateCheckResult` — models the two outcomes of `GET /client/update` in one value so the 204 “no update” semantics are explicit rather than a nil pointer.
Constructors: `NoUpdate()`, `HasUpdate(UpdateAvailable)`.
- `TelemetryReport` — a single device lifecycle event report.

Payload properties (`payload.go`)

- `PayloadProperties` struct (`FileHash`, `FileSize`, `MetadataHash`, `MetadataSize`) — typed form of the four mandatory AOSP `update_engine` headers.
 - `Headers()` `map[string]string` — render to the verbatim `applyPayload` header map.
 - `Validate()` `error` — non-empty hashes, non-negative sizes.
- `ParsePayloadProperties(map[string]string) (PayloadProperties, error)` — parse a header map back to the typed struct.
- Header-key constants: `HeaderFileHash`, `HeaderFileSize`, `HeaderMetadataHash`, `HeaderMetadataSize`.

Validation (`validate.go`)

- `ValidateSHA256(string) error` — exactly 64 lowercase hex chars.
- `ValidateArtifactMeta(ArtifactMeta) error` — SHA-256, optional SHA-512, positive size, valid `OSType`, non-empty board/version/signature.

- `ValidateUpdateCheckRequest(UpdateCheckRequest)` error.
- `ValidateTelemetryReport(TelemetryReport)` error — required ids, valid event, `0..100` progress, non-zero timestamp.

Sentinel errors (`errors.Is-matchable`)

`ErrInvalidEnum, ErrInvalidSHA256, ErrMissingField, ErrInvalidValue` (`validate.go`); `ErrMissingPayloadHeader` (`payload.go`).

Usage

```
package main

import (
    "fmt"

    otaprotocol "github.com/HelixDevelopment/ota-protocol"
)

func main() {
    meta := otaprotocol.ArtifactMeta{
        SHA256:
            "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855",
        Size:      1024,
        OSType:     otaprotocol.OSAndroid,
        Board:      "rk3588",
        Version:    "1.2.0",
        Signature:  "base64-detached-sig",
    }
    if err := otaprotocol.ValidateArtifactMeta(meta); err != nil {
        panic(err)
    }

    // "no update" (204) vs "update available" (200) is explicit,
    // not a nil pointer.
    res := otaprotocol.NoUpdate()
    fmt.Println(res.UpdateAvailable) // false

    props := otaprotocol.PayloadProperties{
        FileHash: "abc", FileSize: 1024, MetadataHash: "def",
        MetadataSize: 64,
    }
    fmt.Println(props.Headers()[otaprotocol.HeaderFileSize]) //
        "1024"
}
```

Testing

```
cd submodules/ota-protocol
go vet ./...
go test ./...
```

The suite (`enums_test.go`, `payload_test.go`, `types_test.go`, `validate_test.go`) covers: JSON round-trips for every wire struct (`TestArtifactRoundTrip`, `TestReleaseRoundTrip`, `TestUpdateAvailableRoundTrip`, ...); the 204/200 `UpdateCheckResult` semantics (`TestUpdateCheckResultSemantics`); enum validity, marshal/unmarshal **rejection** of invalid and non-string tokens (`TestEnumMarshalInvalidRejected`, `TestEnumUnmarshalInvalidRejected`, `TestEnumUnmarshalNonStringRejected`, `TestEnumRoundTrip`); the exactly-six `TelemetryEvent` set (`TestTelemetryEventExactSet`); payload header render / parse / validate round-trips (`TestPayloadPropertiesHeadersRoundTrip`, `TestParsePayloadProperties`); SHA-256 / metadata / telemetry validation and the descriptiveness of validation errors (`TestValidateSHA256`, `TestValidateArtifactMeta`, `TestValidateTelemetryReport`, `TestValidationErrorsAreDescriptive`).

Reusable building brick

This is a **reusable, independently versioned** Helix OTA building brick (Helix Constitution §11.4.28 — submodules-as-equal-codebase). Consume it via its module path `github.com/HelixDevelopment/ota-protocol`. It is project-not-aware and reusable by any project needing the OTA wire contracts. Universal constitution rules are inherited via this repo's `CLAUDE.md` / `AGENTS.md` (`## INHERITED FROM Helix Constitution`).

Mirrors

- GitHub: <https://github.com/HelixDevelopment/ota-protocol>
- GitLab: <https://gitlab.com/helixdevelopment1/ota-protocol>